Université d'Ottawa
Faculté de génie

École de science informatique
et de génie électrique

uOttawa
L'Université canadienne
Canada's university

University of Ottawa
Faculty of Engineering

School of Electrical Engineering
and Computer Science

CEG 4166 / CSI 4141 – Real-Time Systems Design
SEG 4145 – Real-Time and Embedded Software Design
Winter 2026

Laboratory 2 – FreeRTOS Setup and Basic Motor Control (Open-Loop)

1. Learning Objectives

By the end of this laboratory, students will be able to:

- Configure and run a FreeRTOS-based embedded application.
- Establish a clean and scalable task-based architecture.
- Implement open-loop DC motor control using PWM.
- Control motor direction using GPIO signals.
- Read an analog input (potentiometer) using the ADC.
- Integrate basic user feedback using LEDs and a buzzer.

This laboratory establishes the **base software architecture** that will be reused and extended in subsequent laboratories.

2. Hardware Configuration (Mandatory)

Students must use the following signals and peripherals:

Motor Driver

- **Standby (STB):** PE15 (GPIO output)
- **PWM Speed Control:** PE9 → TIM1_CH1

- **Direction Control:**
    - PB10 → AI1
    - PB11 → AI2

Potentiometer

- **ADC Input:** PC0 (ADC channel)

Buzzer

- **Buzzer Output:** PA0 (GPIO or PWM using TIM2/TIM5)

3. FreeRTOS Task Architecture (Required)

At minimum, **two FreeRTOS tasks must be created**:

3.1 Motor Task

Responsible for:

- Applying the current motor state (STOP / CW / CCW).
- Controlling:
    - PWM duty cycle (speed).
    - Motor direction pins.
    - Standby pin.
- Updating LEDs according to the current state.
- Ensuring motor safety (PWM = 0 in STOP state).

Recommended period: 10–20 ms
Priority: Higher than UI Task

3.2 UI Task

Responsible for:

- Reading the push button (user input).

- Reading the potentiometer via ADC.
- Converting ADC value to a PWM duty cycle (0–100%).
- Requesting state changes.
- Triggering a short buzzer beep on each button press.

Recommended period: 20–50 ms
Priority: Lower than Motor Task

4. System Behavior Requirements

4.1 Initial State (MANDATORY)

At system startup:

- Motor state = **STOP**
- PWM duty cycle = **0**
- **Red LED ON**
- Motor must not rotate

4.2 Motor States

State	Motor	LED
STOP	Disabled (PWM = 0)	Red
CW	Clockwise rotation	Green
CCW	Counter-clockwise rotation	Blue

➡ Only **one LED may be ON at any time**.

4.3 Speed Control (Open-Loop)

- The potentiometer controls motor speed.
- ADC value must be mapped to a PWM duty cycle.

- Speed control is active **only when motor is running**.
- In STOP state, PWM must remain 0 regardless of potentiometer position.

4.4 Buzzer Feedback

- A **single short beep** must be generated **on each button press**.
- The buzzer implementation **must not block task execution**.
- HAL_Delay() inside tasks is strongly discouraged.

5. Implementation Constraints

- FreeRTOS must be used (no super-loop design).
- Tasks must communicate using **safe mechanisms**:
    - task notifications, queues, or protected shared data.
- Code must be written with **future labs in mind** (extensible design).

6. Required Demonstration (In-Lab)

During the demonstration, students must show:

1. FreeRTOS scheduler running.
2. Two active tasks (Motor Task and UI Task).
3. Correct initial state at reset.
4. Potentiometer controlling speed.
5. Correct motor direction.
6. LED behavior matching motor state.
7. Buzzer feedback on button press.

7. Deliverables

7.1 Functional Code

- Fully working FreeRTOS project.
- Clean separation of tasks.
- Correct motor, ADC, LED, and buzzer behavior.

7.2 Task Architecture Diagram

A diagram showing:

- Motor Task
- UI Task
- Data/control flow between tasks
- Hardware interfaces

7.3 Task Priority Explanation (Short)

A brief paragraph explaining:

- Why the Motor Task has higher priority.
- Why the UI Task has lower priority.
- How this choice improves determinism and safety.

8. Evaluation Rubric – Lab 2 (20 pts)

Criterion	**Points**
FreeRTOS tasks and scheduling	6
PWM and motor control	6
ADC speed control	4
LEDs and initial state	2
Buzzer feedback integration	1

Criterion	Points
Code quality	1
Total	**20**

9. Notes to Students

- This lab is **foundational**. Poor design here will impact future labs.
- Blocking delays inside tasks will result in penalties.
- Code readability and structure matter.